

Question 2

[30 Marks]

Given a C++ program file in **Program 2**, which contains a linked list implementation of stack to manage the browsing history of a webpage. The browsing history consists of a stack of URLs. Every time the user navigates within the same website, the URL of the new page is placed at the top of the stack. If the user clicks "back" and then click on a new link, the top-most element on the stack will be overwritten with the new URL. The definition of the class **nodeStack** and **stack** are given in the program file. The variable **top** in the class **stack** is a pointer which points to the top node of the stack. The implementation of the method **createStack()** and **isEmpty()** of the class **stack** are also given in **Program 2**.

Complete the implementation of **push()**, **pop()**, **display()** and **search()** function by appending the appropriate lines of code at the space provided in **Program 2** to accomplish each of the following tasks:

1	//PROGRAM 2
2	class nodeStack
3	{
4	{
5	public:
6	
7	string url;
8	
9	string title;
10	
11	nodeStack* next;
12	nodeStack() {
13	
14	url = "";
15	
16	title = "";
17	}
18	
19	nodeStack(string url, string title) {
20	
21	this->url = url;
22	
23	this->title = title;
24	}
25	

```

26
27 };
28
29 class stack
30 {
31
32     nodeStack* top;
33
34 public:
35     void createStack() {
36         top = NULL;
37     }
38
39     bool isEmpty() {
40         return top == NULL;
41     }
42
43     void push();
44     void pop();
45     void display();
46     void search();
47 };
48
49 void stack::push()
50 {
51     string newUrl, newTitle;
52     nodeStack* node = new nodeStack;
53     if (node == NULL)
54         cout << "Cannot allocate memory" << endl;
55     else
56     {
57         system("CLS");
58         cout << "<<<<<<< Insert Web Pages >>>>>>>" << endl << endl;
59
60         //Task 1 - Write the code to insert new browsing history to the stack.
61         Firstly, ask the
62
63         // user to enter the information of the url and title of the browsing
64         history.
65
66
67
68
69
70
71
72
73
74
75
76
77
78

```

```

79 // Then, insert the new browsing item by using the information
80
81 // entered - 8 MARKS
82
83 //Answer Task 1 start here
84
85 //Task 1 end
86 }
87
88 }
89
90
91 void stack::pop()
92 {
93
94     nodeStack* delNode;
95
96     system("CLS");
97
98     cout << "<<<<<< Remove Web Browsing History >>>>>>" << endl <<
99 endl;
100
101     cout << "URL : " << top->url << endl;
102
103     cout << "Title : " << top->title << endl;
104
105     cout << "<< Press any key to continue >>";
106
107     cin.get();
108
109 //Task 2 - Write the code to remove the history from the stack - 3 MARKS
110
111 //Answer Task 2 start here
112
113 //Task 2 end
114 }
115
116 void stack::display()
117 {
118
119     int num = 0;
120     nodeStack* currNode = top;
121
122     system("CLS");
123
124     cout << "<<<<<< Display Web Browsing History >>>>>>" << endl <<
125 endl;
126
127     //Task 3 - Write the code to display all the browsing history in the
128 stack - 9 MARKS
129
130 //Answer Task 3 start here
131

```

```

132 //Task 3 end
133
134     cout << "<< Press any key to continue >>";
135
136     cin.get();
137 }
138
139 void stack::search()
140 {
141
142     int choice;
143
144     string input;
145
146     string abc;
147
148     nodeStack* currNode = top;
149
150     system("CLS");
151
152     cout << "<<<<<<< Search Browsing History(s) >>>>>>>" << endl;
153
154 //Task 4 - Write the code to search the browsing history in the stack based
155 on title - 10 MARKS
156
157 //Answer Task 4 start here
158
159 //Task 4 end
160
161     cout << "<< Finish searching... Press any key to continue >>";
162
163     cin.get();
164 }
165
166 //To display main menu
167 void dispMenu()
168 {
169
170     system("CLS");
171
172     cout << "WEB PAGE BROWSING HISTORY" << endl
173
174         << "\n\t1. Insert New Page"
175
176         << "\n\t2. Remove Page"
177
178         << "\n\t3. Search Page"
179
180         << "\n\t4. Display Page History"
181
182         << "\n\t5. Exit" << endl;
183
184 }
185
186 int main()

```

```
185
186 {
187     stack webBrowse;
188
189     webBrowse.createStack();
190
191     int choice;
192
193     do
194     {
195
196         dispMenu();
197
198         cout << "\nEnter your choice [1-5]: ";
199
200         cin >> choice;
201
202         cin.ignore();
203
204         switch (choice)
205         {
206             case 1: webBrowse.push();
207
208                 break;
209
210             case 2: webBrowse.pop();
211
212                 break;
213
214             case 3: webBrowse.search();
215
216                 break;
217
218             case 4: webBrowse.display();
219
220                 break;
221
222             default: cout << "\nThank you.. see you again.." << endl;
223
224                 return 0;
225
226         }
227     } while ((choice > 0) && (choice < 5));
228
229     return 0;
230 }
231
232
233
234
235
236
237
238
```

Task 1: Write the code to insert new browsing history to the stack. Firstly, ask the user to enter the information of the url and title of the browsing history. Then, insert the new browsing item by using the information entered. **(8 marks)**

Task 2: Write the code to remove the history from the stack . **(3 marks)**

Task 3: Write the code to display all the browsing history in the stack. **(9 marks)**

Task 4: Write the code to search the browsing history in the stack based on the title. **(10 marks)**

Figure 2(a) and 2(b) show the expected output when the program runs. *Note:* The values in **bold** are input by the user.

```

WEB PAGE BROWSING HISTORY

1. Insert New Page
2. Remove Page
3. Search Page
4. Display Page History
5. Exit

Enter your choice [1-5]: 1

<<<<<< Insert New Page >>>>>>

Page URL : www.google.com
Page Title : Google

WEB PAGE BROWSING HISTORY

1. Insert New Page
2. Remove Page
3. Search Page
4. Display Page History
5. Exit

Enter your choice [1-5]: 1

<<<<<< Insert New Page >>>>>>

Page URL : www.facebook.com
Page Title : Facebook

WEB PAGE BROWSING HISTORY

1. Insert New Page
2. Remove Page
3. Search Page
4. Display Page History
5. Exit

Enter your choice [1-5]: 1

<<<<<< Insert Item >>>>>>

Page URL : www.utm.my
Page Title : UTM

WEB PAGE BROWSING HISTORY

1. Insert New Page
2. Remove Page
3. Search Page
4. Display Page History
5. Exit

Enter your choice [1-5]: 4

<<<<<< Display Item >>>>>>

[1] Page URL : www.google.com
Page Title : Google

[2] Page URL : www.apple.com
Page Title : Apple

[3] Page URL : www.utm.my
Page Title : UTM

<< Press any key to continue >>

WEB PAGE BROWSING HISTORY

1. Insert New Page
2. Remove Page
3. Search Page
4. Display Page History
5. Exit

Enter your choice [1-5]: 3

<<<<<< Search Page(s) >>>>>>

Enter Page Title: UTM

Page URL : www.utm.my
Page Title : UTM

<< Finish searching... Press any key
to continue >>

```

Figure 2(a): The output of the program for Question 2

```
WEB PAGE BROWSING HISTORY

1. Insert New Page
2. Remove Page
3. Search Page
4. Display Page History
5. Exit

Enter your choice [1-5]: 2

<<<<<< Remove Page >>>>>>

Page URL : www.utm.my
Page Title : UTM

<< Press any key to continue >>
```

```
WEB PAGE BROWSING HISTORY

1. Insert New Page
2. Remove Page
3. Search Page
4. Display Page History
5. Exit

Enter your choice [1-5]: 4

<<<<<< Display Page History >>>>>>

[1] Page URL : www.google.com
Page Title : Google

[2] Page URL : www.apple.com
Page Title : Apple

<< Press any key to continue >>
```

Figure 2(b): The output of the program for Question 2